

Idea: Create a CLI based game similar to Quiplash, combined with a ranked choice voting system for the users to vote on their favourite prompt/phrase combos.

- This game would differ from standard Quiplash in that no single user is responsible for writing the immediate prompt, instead, the host system will choose a random prompt from a locally stored file.
- Users would be able to type in their own response to specific blanks in the prompt, once all users have given their response, the users will vote on their favourite using a ranked voting system to ensure a fair score distribution despite the lack of a czar/host directly picking their favourite response.

Systems: This would utilize sockets to communicate with the host user, allowing for all users to send in their responses to the host concurrently without blocking any user. Further, we would also utilize pipes within each user's game processes to allow for communication, reading and writing to the pipe as necessary for voting and inputs without blocking or waiting for other user responses.

- The final calculations and data transmitted will be handled by the host process, which communicates with all of the other machines using sockets.
- The player is only responsible for their own ranked voting and response to the prompt, beyond that, the host process takes all data from the child processes transmissions and returns it as necessary to the child processes to display scores, prompts, etc. Effectively, the host process does almost all of the work and the child processes are effectively just IO devices to the host.

Concurrency: Concurrency is required in this design as otherwise, you would result in constant freezing and hanging as you await for others responses. This falls under Category 2 due to the usage of sockets, and as such, will utilize the select keyword to handle each player process as necessary.

Errors, Leaks, Etc: We will avoid leaks by closing pipes as they finish their use and terminating all player processes only when the host process terminates. Otherwise, the host will have programmed behaviour to handle any individual user disconnecting without ending the game for everybody else, dependent on the stage of the game (ie. no vote from that user, or don't wait for that user's response, etc.).

Game Process:

- A user runs the initial executable, given a choice to make this instance the host or a player.
- The first user will make a host, which others can then connect to from their player instances.
 - The host instance will have the ability to start the game, (potentially) require a password to join, or set a limit to the number of players. Also, it will have the option to remove a given user likely through terminal commands.
- Users playing will have access to a player instance, which allows them to connect to a host to play the game.
- Once the game starts, all player instances are dealt the initial prompt randomly selected by the host instance. They will be given a set amount of time to write a response to the prompt. This will be transmitted to the host as the user finishes.
- The host will aggregate all responses, send them out to the players, and await for the users to begin responding with their ranked voting.
- The player processes will each be given five points to rank their favourite prompts, with a maximum of three going to any specific prompt. This is configurable by the host in an earlier step. This is then transmitted back to the host and stored as the user's score. The game loop restarts at the random initial prompt step.

